



# Resumo Abrangente - Projetos Web API e Background Services

*(Gabriel Kohlrausch)*



## **AVISO DE CONFIDENCIALIDADE**

**ESTE MATERIAL FOI ELABORADO EXCLUSIVAMENTE PARA OS PARTICIPANTES DO WORKSHOP DE C# DO JEITO CERTO, CONDUZIDA PELOS PROFISSIONAIS DA EXIMIA.CO.**

Os conteúdos aqui apresentados refletem discussões realizadas em grupo, incluindo contribuições diretas dos participantes. Portanto, não representam necessariamente a opinião dos instrutores ou de sua empresa. A distribuição, reprodução ou divulgação deste material fora do grupo de participantes do workshop, parcial ou integralmente, não é permitida sem autorização prévia expressa.

**Pedimos respeito à confidencialidade do conteúdo e ao ambiente seguro de troca construído ao longo do Workshop de C# do Jeito Certo.**

# ABSTRACT

A reunião inicia com uma troca de experiências sobre modernização de legado. Gabriel introduz conceitos fundamentais do ASP.NET Core, focando no gerenciamento seguro de configurações via `appsettings`, variáveis de ambiente e User Secrets, além de boas práticas de estruturação de logs com Serilog.

O foco avança para segurança, onde Gabriel detalha a distinção entre autenticação e autorização. É discutida a implementação de OAuth2 e OpenID Connect utilizando Keycloak, além da configuração de políticas de acesso, Claims e JWT, visando substituir validações manuais por padrões de mercado.

Gabriel explora o processamento em segundo plano com `IHostedService` e `BackgroundService`, alertando sobre ciclos de vida e injeção de dependência. Também são abordados Middlewares personalizados para tratamento de erros e estratégias de multi-tenancy para isolamento de dados por cliente.

Por fim, são apresentadas abordagens arquiteturais modernas. Gabriel demonstra o uso do FastEndpoints e do Workflow Core para orquestração de Sagas. A sessão encerra defendendo a simplificação de projetos através de Monorepos, reduzindo a complexidade de soluções com excesso de camadas.

# SUMÁRIO

|                                                          |           |
|----------------------------------------------------------|-----------|
| 1. O FLUXO DE DESENVOLVIMENTO E A REVOLUÇÃO DA IA        | 5         |
| 2. ARQUITETURA DO ASP.NET CORE E PIPELINE DE MIDDLEWARES | 5         |
| 3. PADRÕES DE DESIGN: VERTICAL SLICE E REPR              | 5         |
| 4. GESTÃO AVANÇADA DE INJEÇÃO DE DEPENDÊNCIA (DI)        | 6         |
| 5. PROCESSAMENTO ASSÍNCRONO E BACKGROUND SERVICES        | 6         |
| 6. ESTRATÉGIAS DE MULTI-TENANCY (SAAS)                   | 6         |
| 7. TRATAMENTO DE ERROS E OBSERVABILIDADE                 | 7         |
| <b>INSIGHTS E TAKEAWAYS PRÁTICOS</b>                     | <b>8</b>  |
| 1. PRODUTIVIDADE E INTELIGÊNCIA ARTIFICIAL               | 8         |
| 2. ARQUITETURA E DESIGN DE CÓDIGO                        | 8         |
| 3. ASP.NET CORE E PERFORMANCE                            | 8         |
| 4. INJEÇÃO DE DEPENDÊNCIA (DI) E CICLO DE VIDA           | 9         |
| 5. PROCESSAMENTO ASSÍNCRONO E RESILIÊNCIA                | 9         |
| 6. ESTRATÉGIAS DE MULTI-TENANCY (SAAS)                   | 9         |
| 7. CONFIABILIDADE E PADRONIZAÇÃO                         | 10        |
| <b>GUIA DE ESTUDOS PRÁTICO</b>                           | <b>11</b> |
| 1. PRODUTIVIDADE E INTELIGÊNCIA ARTIFICIAL               | 11        |
| 2. ARQUITETURA E DESIGN DE CÓDIGO                        | 11        |
| 3. ASP.NET CORE E PERFORMANCE                            | 11        |
| 4. INJEÇÃO DE DEPENDÊNCIA (DI) E CICLO DE VIDA           | 12        |
| 5. PROCESSAMENTO ASSÍNCRONO E RESILIÊNCIA                | 12        |
| 6. ESTRATÉGIAS DE MULTI-TENANCY (SAAS)                   | 12        |
| 7. CONFIABILIDADE E PADRONIZAÇÃO                         | 13        |
| <b>FRASES DE IMPACTO</b>                                 | <b>14</b> |
| MÓDULO 1: FUNDAMENTOS E ARQUITETURA ASP.NET CORE         | 14        |
| MÓDULO 2: APIS E PERFORMANCE                             | 14        |
| MÓDULO 3: INJEÇÃO DE DEPENDÊNCIA (DI)                    | 15        |
| MÓDULO 4: PROCESSAMENTO ASSÍNCRONO E BACKGROUND SERVICES | 15        |
| MÓDULO 5: ESTRATÉGIAS DE MULTI-TENANCY                   | 15        |
| MÓDULO 6: MENTALIDADE E NOVAS FERRAMENTAS                | 16        |
| DICA DE ESTUDO PRÁTICO:                                  | 16        |
| <b>GLOSSÁRIO ABRANGENTE</b>                              | <b>17</b> |
| <b>TABELA DE RELACIONAMENTO ONTOLÓGICO</b>               | <b>24</b> |
| <b>MOMENTOS MAIS INTERESSANTES DO ENCONTRO</b>           | <b>28</b> |
| <b>PERGUNTAS E RESPOSTAS DO ENCONTRO</b>                 | <b>31</b> |
| <b>AGORA É COM VOCÊ</b>                                  | <b>37</b> |
| COMPARTILHE SUA JORNADA                                  | 37        |



# 1. O FLUXO DE DESENVOLVIMENTO E A REVOLUÇÃO DA IA

A reunião inicia com uma reflexão sobre a produtividade moderna. Introduz-se o conceito de **Vibe Coding**, onde o desenvolvedor atua mais como um orquestrador de intenções e revisor de código gerado por IA (utilizando ferramentas como o **Cursor IDE**), em vez de apenas um digitador de sintaxe. A discussão enfatiza que, embora a IA acelere o desenvolvimento "Greenfield" (projetos do zero), a senioridade e o conhecimento de design continuam sendo o "último guardião" para evitar débitos técnicos em sistemas legados ou complexos.

## 2. ARQUITETURA DO ASP.NET CORE E PIPELINE DE MIDDLEWARES

Foi revisitada a base do motor do ASP.NET Core:

- **Kestrel vs. Proxy Reverso:** Reforçou-se a obrigatoriedade de não expor o Kestrel diretamente à internet. O uso de um Proxy Reverso (IIS, Nginx, Apache ou Cloudflare) é essencial para *offload* de SSL, *rate limiting* e segurança.
- **Pipeline de Middlewares:** A ordem de execução dos filtros e componentes é crítica. Inverter a ordem entre `UseAuthentication` e `UseAuthorization`, por exemplo, compromete toda a segurança da aplicação.
- **Minimal APIs vs. Controller-based:** Discutiu-se o ganho de performance das Minimal APIs (que utilizam menos *reflection* e são compatíveis com Native AOT), recomendando-as para microsserviços enxutos, enquanto as Controllers permanecem viáveis para monolitos com roteamento complexo.

## 3. PADRÕES DE DESIGN: VERTICAL SLICE E REPR

Houve uma forte defesa da migração das arquiteturas em camadas tradicionais (pastas separadas por Infra, Domínio e Aplicação) para o **Vertical Slice Architecture**.

- **Padrão REPR (Request-Endpoint-Response):** Propõe que cada endpoint seja uma classe autônoma contendo seu contrato de entrada e saída. Isso

reduz o acoplamento e o "custo da mudança", permitindo que alterações em uma funcionalidade não afetem outras partes do sistema.

## 4. GESTÃO AVANÇADA DE INJEÇÃO DE DEPENDÊNCIA (DI)

Um dos pontos mais técnicos tratou dos ciclos de vida (Lifetimes):

- **Captive Dependency:** O perigo de um objeto de vida longa (Singleton) segurar um objeto de vida curta (Scoped), como um `DbContext`.
- **Validação em Build:** A recomendação de ativar `ValidateScopes` e `ValidateOnBuild` no `Program.cs` para detectar erros de injeção logo na inicialização.
- **Scope Factory em Background Services:** Como criar escopos manuais dentro de tarefas de segundo plano para garantir o descarte correto de recursos.

## 5. PROCESSAMENTO ASSÍNCRONO E BACKGROUND SERVICES

A aula explorou o uso de **Background Services** para tarefas como consumo de filas, sincronizações e *warm-up* de dados.

- **System.Threading.Channels:** Apresentado como uma alternativa robusta para comunicação *producer/consumer* dentro do mesmo processo.
- **Workflow Core:** Sugerido para orquestrações complexas de tarefas que exigem persistência de estado e resiliência a falhas, permitindo que a API libere a requisição rapidamente enquanto o processo ocorre em background.

## 6. ESTRATÉGIAS DE MULTI-TENANCY (SAAS)

O grupo debateu como lidar com múltiplos clientes (tenants) em uma única aplicação:

- **Identificação:** Via Headers, Subdomínios ou Claims de JWT.

- **Isolamento de Dados:** Comparação entre o uso de uma única base com filtros globais (Global Query Filters no EF Core) e a estratégia de bases de dados isoladas por cliente através de um **DbContextFactory** dinâmico.
- **Segurança:** A necessidade de validar no backend se o usuário autenticado realmente possui permissão para acessar o **TenantId** enviado na requisição.

## 7. TRATAMENTO DE ERROS E OBSERVABILIDADE

A recomendação final foca na padronização:

- Uso da **RFC 7807 (Problem Details)** para retornos de erro consistentes.
- Implementação de um **Global Exception Handler** para evitar blocos **try-catch** repetitivos nas Controllers.
- Adoção do **Serilog** para logs estruturados e enriquecidos com contexto, facilitando a depuração em ambientes distribuídos.

**Conclusão Estratégica:** A reunião encerra com a indicação do livro "*Righting Software*", de Juval Löwy, incentivando os desenvolvedores a projetarem sistemas baseados na **volatilidade** dos componentes, preparando o software para mudanças inevitáveis com o menor custo possível.



# INSIGHTS E TAKEAWAYS PRÁTICOS



## 1. PRODUTIVIDADE E INTELIGÊNCIA ARTIFICIAL

- **Vibe Coding:** O desenvolvimento está migrando da escrita manual de sintaxe para a orquestração de intenções via IA. O uso do **Cursor IDE** é um divisor de águas na velocidade de entrega.
- **O Papel do Sênior:** Em um mundo assistido por IA, o valor do desenvolvedor sênior reside menos na escrita do código e mais na **validação do design** e na prevenção de débitos técnicos.

## 2. ARQUITETURA E DESIGN DE CÓDIGO

- **Custo da Mudança:** Arquiteturas tradicionais em camadas (N-Tier) aumentam o risco e o custo de alterações. A **Vertical Slice Architecture** é preferível por isolar funcionalidades e reduzir o acoplamento.
- **Padrão REPR (Request-Endpoint-Response):** Agrupar o contrato de entrada, a lógica e o retorno no mesmo arquivo (especialmente com Minimal APIs) minimiza a carga cognitiva e facilita a manutenção.
- **Design Baseado em Volatilidade:** Projete seu sistema focando nos componentes que têm maior probabilidade de mudar, isolando-os para proteger o restante do sistema (conceito de Juval Löwy).

## 3. ASP.NET CORE E PERFORMANCE

- **Kestrel sob custódia:** Nunca exponha o Kestrel diretamente. O uso de um **Proxy Reverso** (Nginx, IIS, Cloudflare) é obrigatório para garantir segurança e recursos avançados como *rate limiting*.
- **Minimal APIs vs. Controllers:** Use Minimal APIs para performance bruta e compatibilidade com **Native AOT**. Mantenha Controllers apenas se a complexidade de roteamento e filtros de um monolito legado exigir.

- **A importância da ordem:** No ASP.NET Core, a ordem dos Middlewares no `Program.cs` define o sucesso ou falha da requisição. Erros comuns de segurança nascem da inversão entre Autenticação e Autorização.

## 4. INJEÇÃO DE DEPENDÊNCIA (DI) E CICLO DE VIDA

- **Captive Dependency:** É um erro crítico de produção onde um objeto Singleton "sequestra" um Scoped (como o `DbContext`), causando falhas de concorrência.
- **Fail Fast:** Ative sempre `ValidateScopes` e `ValidateOnBuild` no bootstrap da aplicação para que erros de DI sejam detectados na compilação, e não em tempo de execução.
- **Escopos em Background:** Em `BackgroundServices`, nunca injete serviços Scoped diretamente no construtor. Use o `IServiceScopeFactory` para criar escopos manuais e garantir o descarte de recursos.

## 5. PROCESSAMENTO ASSÍNCRONO E RESILIÊNCIA

- **Channels para Produtor/Consumidor:** O `System.Threading.Channels` é a ferramenta nativa ideal para gerenciar filas em memória e *back pressure* dentro da aplicação sem a complexidade de um broker externo inicial.
- **Workflow Core:** Para processos complexos que dependem de estados ou aprovações externas, prefira motores de workflow em vez de lógicas manuais complexas em Background Services.

## 6. ESTRATÉGIAS DE MULTI-TENANCY (SAAS)

- **Segurança no Tenant:** O `TenantId` vindo do header deve ser sempre validado contra as `claims` do JWT do usuário no backend para evitar que um cliente acesse dados de outro apenas trocando um ID.
- **Isolamento de Dados:**
  - **Banco Único:** Use *Global Query Filters* do EF Core para transparência.
  - **Bancos Separados:** Use um `DbContextFactory` dinâmico que resolva a string de conexão em tempo de execução através de um middleware.

## 7. CONFIABILIDADE E PADRONIZAÇÃO

- **Problem Details (RFC 7807):** Pare de criar seus próprios objetos de erro. Use o padrão da RFC para que o front-end e o back-end falem a mesma língua.
- **Logs Estruturados:** Use **Serilog** para enriquecer logs com propriedades (como **TenantId** ou **CorrelationId**), permitindo buscas granulares em ferramentas de observabilidade.
- **Bootstrap Robusto:** Utilize blocos **try-catch** no **Program.cs** para garantir que falhas na subida da aplicação sejam logadas corretamente antes do processo morrer.

# GUIA DE ESTUDOS PRÁTICO



## 1. PRODUTIVIDADE E INTELIGÊNCIA ARTIFICIAL

- **Vibe Coding:** O desenvolvimento está migrando da escrita manual de sintaxe para a orquestração de intenções via IA. O uso do **Cursor IDE** é um divisor de águas na velocidade de entrega.
- **O Papel do Sênior:** Em um mundo assistido por IA, o valor do desenvolvedor sênior reside menos na escrita do código e mais na **validação do design** e na prevenção de débitos técnicos.

## 2. ARQUITETURA E DESIGN DE CÓDIGO

- **Custo da Mudança:** Arquiteturas tradicionais em camadas (N-Tier) aumentam o risco e o custo de alterações. A **Vertical Slice Architecture** é preferível por isolar funcionalidades e reduzir o acoplamento.
- **Padrão REPR (Request-Endpoint-Response):** Agrupar o contrato de entrada, a lógica e o retorno no mesmo arquivo (especialmente com Minimal APIs) minimiza a carga cognitiva e facilita a manutenção.
- **Design Baseado em Volatilidade:** Projete seu sistema focando nos componentes que têm maior probabilidade de mudar, isolando-os para proteger o restante do sistema (conceito de Juval Löwy).

## 3. ASP.NET CORE E PERFORMANCE

- **Kestrel sob custódia:** Nunca exponha o Kestrel diretamente. O uso de um **Proxy Reverso** (Nginx, IIS, Cloudflare) é obrigatório para garantir segurança e recursos avançados como *rate limiting*.
- **Minimal APIs vs. Controllers:** Use Minimal APIs para performance bruta e compatibilidade com **Native AOT**. Mantenha Controllers apenas se a complexidade de roteamento e filtros de um monolito legado exigir.
- **A importância da ordem:** No ASP.NET Core, a ordem dos Middlewares no `Program.cs` define o sucesso ou falha da requisição. Erros comuns de segurança nascem da inversão entre Autenticação e Autorização.

## 4. INJEÇÃO DE DEPENDÊNCIA (DI) E CICLO DE VIDA

- **Captive Dependency:** É um erro crítico de produção onde um objeto Singleton "sequestra" um Scoped (como o DbContext), causando falhas de concorrência.
- **Fail Fast:** Ative sempre `ValidateScopes` e `ValidateOnBuild` no bootstrap da aplicação para que erros de DI sejam detectados na compilação, e não em tempo de execução.
- **Escopos em Background:** Em `BackgroundServices`, nunca injete serviços Scoped diretamente no construtor. Use o `IServiceScopeFactory` para criar escopos manuais e garantir o descarte de recursos.

## 5. PROCESSAMENTO ASSÍNCRONO E RESILIÊNCIA

- **Channels para Produtor/Consumidor:** O `System.Threading.Channels` é a ferramenta nativa ideal para gerenciar filas em memória e *back pressure* dentro da aplicação sem a complexidade de um broker externo inicial.
- **Workflow Core:** Para processos complexos que dependem de estados ou aprovações externas, prefira motores de workflow em vez de lógicas manuais complexas em Background Services.

## 6. ESTRATÉGIAS DE MULTI-TENANCY (SAAS)

- **Segurança no Tenant:** O `TenantId` vindo do header deve ser sempre validado contra as `claims` do JWT do usuário no backend para evitar que um cliente acesse dados de outro apenas trocando um ID.
- **Isolamento de Dados:**
  - **Banco Único:** Use `Global Query Filters` do EF Core para transparência.
  - **Bancos Separados:** Use um `DbContextFactory` dinâmico que resolva a string de conexão em tempo de execução através de um middleware.

## 7. CONFIABILIDADE E PADRONIZAÇÃO

- **Problem Details (RFC 7807):** Pare de criar seus próprios objetos de erro. Use o padrão da RFC para que o front-end e o back-end falem a mesma língua.
- **Logs Estruturados:** Use **Serilog** para enriquecer logs com propriedades (como **TenantId** ou **CorrelationId**), permitindo buscas granulares em ferramentas de observabilidade.
- **Bootstrap Robusto:** Utilize blocos **try-catch** no **Program.cs** para garantir que falhas na subida da aplicação sejam logadas corretamente antes do processo morrer.

# FRASES DE IMPACTO



## MÓDULO 1: FUNDAMENTOS E ARQUITETURA ASP.NET CORE

### 1. Por que não devemos expor o servidor Kestrel diretamente à internet?

**Resposta:** O Kestrel não possui recursos avançados de segurança e gerenciamento de tráfego. O uso de um **Proxy Reverso** (como Nginx, IIS ou Cloudflare) é obrigatório para realizar o *offload* de SSL, implementar *rate limiting*, proteger contra ataques e gerenciar melhor as conexões estáticas.

**2. Qual a importância da ordem dos Middlewares no Program.cs? Resposta:** O ASP.NET Core funciona como uma "esteira" (pipeline). A ordem define como a requisição é processada. Por exemplo, o **UseAuthentication** deve sempre vir antes do **UseAuthorization**, pois você não pode autorizar alguém que ainda não foi identificado. Inverter essa ordem quebra a segurança da aplicação.

**3. O que é Vertical Slice Architecture e qual sua vantagem sobre a arquitetura em camadas tradicional? Resposta:** Ao contrário da arquitetura em camadas (N-Tier), que separa o código por pastas como "Infra", "Domain" e "App", a **Vertical Slice** organiza o código por funcionalidades (ex: "CriarPedido", "ListarProdutos"). Isso reduz o acoplamento, diminui a carga cognitiva e baixa o "custo da mudança", pois as alterações ficam contidas em um único bloco funcional.

**4. Como o padrão REPR (Request-Endpoint-Response) se aplica às Minimal APIs? Resposta:** O REPR propõe que cada endpoint seja uma classe independente que contém seu contrato de entrada (Request), a lógica de negócio (Endpoint) e o contrato de saída (Response). Isso elimina Controllers gigantes e centraliza tudo o que uma funcionalidade precisa em um só lugar.

## MÓDULO 2: APIS E PERFORMANCE

**5. Qual a diferença técnica entre Minimal APIs e APIs baseadas em Controllers? Resposta:** Minimal APIs são mais leves, possuem menos *boilerplate* e não utilizam *reflection* pesado para o roteamento, o que as torna mais performáticas e compatíveis com o **Native AOT** (compilação nativa). Controllers são recomendadas para cenários legados ou monolitos com regras de roteamento extremamente complexas.

**6. O que é a RFC 7807 (Problem Details) e por que usá-la? Resposta:** É um padrão para retornos de erro em APIs HTTP. Em vez de retornar mensagens de texto simples ou objetos de erro customizados, o uso de `ProblemDetails` garante que o front-end e o back-end utilizem uma estrutura universal e previsível para tratar erros (contendo tipo, título, status e detalhes).

## MÓDULO 3: INJEÇÃO DE DEPENDÊNCIA (DI)

**7. O que é uma "Captive Dependency" (Dependência Cativa)? Resposta:** Ocorre quando um serviço com tempo de vida mais longo injeta um serviço com tempo de vida mais curto. Exemplo: Um serviço **Singleton** que recebe um **Scoped** (como o `DbContext`). O Singleton manterá o Scoped vivo para sempre, impedindo o descarte de conexões com o banco e gerando erros de concorrência e memória.

**8. Como garantir que as injeções de dependência estão corretas no início da aplicação? Resposta:** Deve-se configurar o `DefaultServiceProvider` no bootstrap da aplicação ativando `ValidateScopes` e `ValidateOnBuild` como `true`. Isso faz com que a aplicação apresente um erro imediato ao subir se houver alguma configuração de DI inválida.

## MÓDULO 4: PROCESSAMENTO ASSÍNCRONO E BACKGROUND SERVICES

**9. Como utilizar serviços Scoped (como o DbContext) dentro de um BackgroundService? Resposta:** Como o `BackgroundService` é um Singleton, você não pode injetar serviços Scoped no construtor. A solução é injetar o `IServiceScopeFactory`, criar um escopo manual dentro do método `ExecuteAsync` através de um bloco `using`, e então solicitar o serviço desejado a partir desse escopo.

**10. Quando utilizar System.Threading.Channels em vez de um Broker de Mensageria (como RabbitMQ)? Resposta:** Os **Channels** são ideais para filas em memória dentro do mesmo processo da aplicação. Devem ser usados para gerenciar *back pressure* e paralelismo interno quando você não precisa da persistência e da distribuição que um broker externo oferece.

## MÓDULO 5: ESTRATÉGIAS DE MULTI-TENANCY

**11. Quais as duas principais abordagens para isolamento de dados em sistemas SaaS? Resposta:**



- **Shared Database:** Todos os clientes dividem a mesma tabela, diferenciados por uma coluna `TenantId` (geralmente controlado via *Global Query Filters* no EF Core).
- **Database per Tenant:** Cada cliente possui seu próprio banco de dados físico. Isso exige um `IDbContextFactory` dinâmico que troca a string de conexão baseada no identificador do cliente na requisição.

## MÓDULO 6: MENTALIDADE E NOVAS FERRAMENTAS

### 12. O que é "Vibe Coding" e como ele altera o papel do desenvolvedor?

**Resposta:** É o fluxo de desenvolvimento focado em intenções e revisão, onde ferramentas de IA (como o **Cursor IDE**) escrevem a maior parte do código. O desenvolvedor deixa de ser um "digitador de sintaxe" para se tornar um arquiteto e validador, focando na qualidade do design e na resolução de problemas complexos.

### 13. Qual o conceito principal do livro "Righting Software" aplicado na reunião?

**Resposta:** O design de software deve ser baseado na **volatilidade**. Em vez de organizar o sistema apenas por funções, deve-se identificar as partes que mais mudam e criar isolamentos (encapsulamento) ao redor delas, protegendo o restante da aplicação contra mudanças frequentes.

## DICA DE ESTUDO PRÁTICO:

Tente implementar um projeto simples utilizando **Minimal API**, **Vertical Slice Architecture** e ative as validações de **Injeção de Dependência**. Use o **Serilog** para logar o `TenantId` em cada requisição simulada.

# GLOSSÁRIO ABRANGENTE



| Elemento                          | Descrição do Conceito                                                                                                                                                                                                   |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>API Gateway</b>                | Componente de infraestrutura que atua como ponto de entrada único para um conjunto de microsserviços, gerenciando roteamento, autenticação e políticas de tráfego.                                                      |
| <b>Async / Await</b>              | Padrão de programação assíncrona no C# que permite que threads sejam liberadas durante operações de I/O (entrada/saída), aumentando drasticamente a escalabilidade da aplicação.                                        |
| <b>Background Service</b>         | Classe no .NET que herda de <code>BackgroundService</code> ou implementa <code>IHostedService</code> , utilizada para executar tarefas de longa duração em segundo plano, fora do ciclo de vida de uma requisição HTTP. |
| <b>BFF (Backend for Frontend)</b> | Padrão arquitetural onde um serviço específico é criado para atender às necessidades de uma interface de usuário particular, reduzindo o acoplamento entre o front-end e múltiplos microsserviços.                      |
| <b>Bounded Context</b>            | Conceito central do DDD (Domain-Driven Design) que delimita                                                                                                                                                             |

| <b>Elemento</b>           | <b>Descrição do Conceito</b>                                                                                                                                                           |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                           | os limites lógicos onde um modelo de domínio específico é aplicável, evitando que conceitos se misturem em sistemas grandes.                                                           |
| <b>Captive Dependency</b> | Erro de configuração em Injeção de Dependência onde um serviço com tempo de vida longo (Singleton) mantém uma referência a um serviço de tempo curto (Scoped), impedindo seu descarte. |
| <b>Claims</b>             | Declarações de atributos de um usuário (como e-mail, permissões ou ID do inquilino) contidas dentro de um token de segurança, como o JWT.                                              |
| <b>Cursor IDE</b>         | Editor de código baseado no VS Code que integra Inteligência Artificial de forma nativa para auxiliar na escrita, refatoração e geração de código em tempo real.                       |
| <b>Dapper</b>             | Micro-ORM para .NET focado em performance, que permite executar consultas SQL puras e mapear os resultados para objetos de forma extremamente rápida.                                  |

| Elemento                               | Descrição do Conceito                                                                                                                                                                   |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>EF Core (Entity Framework Core)</b> | ORM (Object-Relational Mapper) oficial da Microsoft que abstrai o acesso ao banco de dados, permitindo trabalhar com dados usando objetos e LINQ.                                       |
| <b>Global Exception Handler</b>        | Middleware ou filtro centralizado que captura e trata todas as exceções não tratadas da aplicação, garantindo retornos padronizados e log de erros eficiente.                           |
| <b>Global Query Filter</b>             | Recurso do EF Core que aplica automaticamente filtros de consulta (como <code>Where IsDeleted = false</code> ou <code>TenantId = X</code> ) em todas as queries executadas no contexto. |
| <b>Injeção de Dependência (DI)</b>     | Padrão de design que permite que classes recebam suas dependências de uma fonte externa em vez de criá-las internamente, facilitando testes e desacoplamento.                           |
| <b>JWT (JSON Web Token)</b>            | Padrão de mercado para representar declarações de segurança entre duas partes, amplamente utilizado para autenticação e autorização em APIs.                                            |

| <b>Elemento</b>                         | <b>Descrição do Conceito</b>                                                                                                                                    |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Kestrel</b>                          | Servidor web multiplataforma, leve e de alta performance, padrão do ASP.NET Core, projetado para processar requisições HTTP de forma rápida.                    |
| <b>LINQ (Language Integrated Query)</b> | Conjunto de recursos do C# que permite realizar consultas em coleções de dados (objetos, bancos de dados, XML) usando uma sintaxe integrada à linguagem.        |
| <b>LinqKit</b>                          | Biblioteca que estende as funcionalidades do LINQ, permitindo a construção dinâmica de predicados e expressões complexas para consultas em banco de dados.      |
| <b>Middleware</b>                       | Componente de software montado em um pipeline (esteira) no ASP.NET Core para manipular requisições e respostas HTTP de forma sequencial.                        |
| <b>Minimal API</b>                      | Abordagem simplificada no ASP.NET Core para criar APIs HTTP com o mínimo de código, eliminando a necessidade da estrutura tradicional de Controllers e Classes. |

| Elemento                          | Descrição do Conceito                                                                                                                                               |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Multi-tenancy</b>              | Arquitetura de software onde uma única instância de aplicação atende a múltiplos clientes (tenants), mantendo o isolamento de dados entre eles.                     |
| <b>Native AOT (Ahead-of-Time)</b> | Tecnologia que compila código .NET diretamente em código de máquina nativo antes da execução, resultando em inicialização mais rápida e menor uso de memória.       |
| <b>Problem Details (RFC 7807)</b> | Especificação padronizada para formatos de resposta de erro em APIs HTTP, contendo detalhes legíveis por humanos e máquinas sobre o problema ocorrido.              |
| <b>Proxy Reverso</b>              | Servidor (como Nginx ou IIS) que fica à frente da aplicação para gerenciar balanceamento de carga, terminação SSL e proteção de borda.                              |
| <b>REPR Pattern</b>               | Padrão <i>Request-Endpoint-Response</i> , que organiza a lógica de APIs em torno de endpoints específicos em vez de agrupar múltiplas ações em um único Controller. |
| <b>Righting Software</b>          | Título do livro de Juval Löwy que defende um design de sistemas                                                                                                     |

| <b>Elemento</b>                  | <b>Descrição do Conceito</b>                                                                                                                                       |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                  | baseado na decomposição por volatilidade em vez de decomposição funcional.                                                                                         |
| <b>Scoped (Ciclo de Vida)</b>    | Tempo de vida de um objeto na Injeção de Dependência onde uma nova instância é criada para cada requisição HTTP e descartada ao final desta.                       |
| <b>Serilog</b>                   | Biblioteca de log estruturado para .NET que permite enriquecer mensagens com propriedades e enviá-las para diversos destinos (console, arquivos, bancos de dados). |
| <b>Singleton (Ciclo de Vida)</b> | Tempo de vida de um objeto na Injeção de Dependência onde uma única instância é criada para toda a duração do processo da aplicação.                               |
| <b>System.Threading.Channels</b> | Biblioteca nativa do .NET para implementação de padrões de produtor/consumidor, permitindo troca de dados segura entre threads de forma performática.              |
| <b>Transient (Ciclo de Vida)</b> | Tempo de vida de um objeto na Injeção de Dependência onde uma                                                                                                      |

| <b>Elemento</b>                    | <b>Descrição do Conceito</b>                                                                                                                                      |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                    | nova instância é criada toda vez que o serviço é solicitado.                                                                                                      |
| <b>Vertical Slice Architecture</b> | Estilo arquitetural que organiza o código por "fatias" de funcionalidade (funcionalidades completas de ponta a ponta) em vez de camadas técnicas horizontais.     |
| <b>Vibe Coding</b>                 | Termo moderno que descreve um fluxo de trabalho onde o desenvolvedor foca na intenção e no design de alto nível, enquanto a IA cuida da implementação da sintaxe. |
| <b>Volatility-based Design</b>     | Abordagem de engenharia onde o sistema é projetado isolando componentes que possuem maior probabilidade de sofrer mudanças frequentes (volatilidade).             |
| <b>Workflow Core</b>               | Motor de fluxo de trabalho leve e persistente para .NET, utilizado para gerenciar processos complexos, assíncronos e que exigem controle de estado.               |



# TABELA DE RELACIONAMENTO ONTOLÓGICO



| Conceito A                  | Conceito B    | Detalhamento da Relação                                                                                                     |
|-----------------------------|---------------|-----------------------------------------------------------------------------------------------------------------------------|
| Kestrel                     | Proxy Reverso | O Kestrel processa as requisições, enquanto o Proxy Reverso (Nginx/IIS) protege a borda e gerencia SSL e tráfego.           |
| Minimal API                 | Native AOT    | Minimal APIs evitam <i>reflection</i> pesado, tornando-as o modelo ideal para compilação nativa (AOT) e alta performance.   |
| Middleware                  | Pipeline HTTP | Middlewares são os filtros individuais que, juntos e em ordem específica, compõem a esteira de processamento da requisição. |
| Vertical Slice Architecture | REPR Pattern  | O REPR é a unidade tática da Vertical Slice; cada fatia funcional é implementada como um trio de                            |

| Conceito A                    | Conceito B                  | Detalhamento da Relação                                                                                                            |
|-------------------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                               |                             | Request-Endpoint-Response.                                                                                                         |
| <b>Injeção de Dependência</b> | <b>Captive Dependency</b>   | Uma configuração incorreta de ciclos de vida na Injeção de Dependência resulta no "sequestro" de objetos Scoped por Singletons.    |
| <b>Background Service</b>     | <b>IServiceScopeFactory</b> | Como o Background Service é um Singleton, ele depende da Factory para criar escopos manuais e acessar serviços Scoped (como o DB). |
| <b>Multi-tenancy</b>          | <b>Global Query Filter</b>  | Em bancos de dados compartilhados, o filtro global do EF Core garante que um cliente (tenant) nunca acesse dados de outro.         |
| <b>Vibe Coding</b>            | <b>Cursor IDE</b>           | O Cursor IDE é a ferramenta técnica que materializa a filosofia de <i>Vibe Coding</i> (desenvolvimento                             |

| <b>Conceito A</b>                     | <b>Conceito B</b>                     | <b>Detalhamento da Relação</b>                                                                                                         |
|---------------------------------------|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|                                       |                                       | guiado por intenção e IA).                                                                                                             |
| <b>Juval Löwy (Righting Software)</b> | <b>Design Baseado em Volatilidade</b> | A obra de Löwy fundamenta a teoria de que o software deve ser decomposto pelas suas partes que mais mudam (volatilidade).              |
| <b>System.Threading.Channels</b>      | <b>Producer/Consumer Pattern</b>      | Os Channels fornecem a estrutura de dados necessária para implementar a comunicação assíncrona entre quem produz e quem consome dados. |
| <b>Problem Details (RFC 7807)</b>     | <b>Global Exception Handler</b>       | O Handler captura erros inesperados e os converte no formato padronizado de Problem Details para o cliente.                            |
| <b>JWT (Claims)</b>                   | <b>Segurança no Multi-tenancy</b>     | As claims de um token JWT devem conter o identificador do tenant para validar se o usuário tem permissão de acesso àquele contexto.    |

| <b>Conceito A</b>                  | <b>Conceito B</b>             | <b>Detalhamento da Relação</b>                                                                                                                     |
|------------------------------------|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Vertical Slice Architecture</b> | <b>Arquitetura em Camadas</b> | A Vertical Slice surge como uma alternativa evolutiva à Arquitetura em Camadas (N-Tier) para reduzir o custo da mudança.                           |
| <b>Dapper</b>                      | <b>Performance de Leitura</b> | O Dapper é utilizado como um complemento ao EF Core para otimizar queries de leitura complexas e reduzir o overhead do ORM.                        |
| <b>Workflow Core</b>               | <b>Resiliência de Estado</b>  | Enquanto Background Services gerenciam tarefas simples, o Workflow Core lida com processos complexos que precisam persistir o estado entre falhas. |

# MOMENTOS MAIS INTERESSANTES DO ENCONTRO



| Título                                               | O Que Foi Dito                                                                                                                                                                                               | Justificativa                                                                                                                                                                                                                      |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A Queda do Último Guardião</b>                    | O palestrante admitiu que era cético em relação ao <i>Vibe Coding</i> e ferramentas de IA, mas mudou de ideia ao ver um projeto funcional e agora se diz "apaixonado" pela produtividade alcançada.          | Este é um marco cultural na reunião. Demonstra que até profissionais seniores e conservadores estão sendo vencidos pela eficiência das IAs generativas (como o Cursor), sinalizando uma mudança irreversível no fluxo de trabalho. |
| <b>O Perigo do Singleton "Sequestrador"</b>          | Foi discutido o conceito de <i>Captive Dependency</i> , onde um Singleton segura um objeto Scoped (como o DbContext), gerando erros que não aparecem em desenvolvimento, mas explodem em produção sob carga. | É o ponto de maior valor técnico preventivo. Ele aborda um erro silencioso e catastrófico de arquitetura, ensinando o desenvolvedor a diagnosticar problemas de concorrência antes mesmo de eles ocorrerem.                        |
| <b>A Morte da Arquitetura em Camadas Tradicional</b> | Houve uma crítica contundente ao modelo N-Tier (Infra, Domain, App) em favor da <i>Vertical Slice Architecture</i> ,                                                                                         | Este momento desafia o <i>status quo</i> de muitos desenvolvedores .NET que seguem o modelo de camadas por hábito.                                                                                                                 |

| <b>Título</b>                                     | <b>O Que Foi Dito</b>                                                                                                                                                                             | <b>Justificativa</b>                                                                                                                                                                                                           |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                   | focando no "Custo da Mudança" e no risco de acoplamento.                                                                                                                                          | A discussão foca em resultados de negócio (manutenibilidade) em vez de purismo estético de pastas.                                                                                                                             |
| <b>O Erro do "Falso Sucesso" (HTTP 200)</b>       | O palestrante manifestou indignação ("chega a dar nervoso") com APIs que retornam Status Code 200 para operações que falharam, contendo uma mensagem de erro no corpo.                            | Este momento reforça a ética e o profissionalismo no uso de protocolos. É um lembrete crucial de que a conformidade com padrões (como as RFCs) é o que diferencia um sistema amador de um sistema de nível empresarial.        |
| <b>Design Orientado à Volatilidade</b>            | A citação ao livro <i>"Righting Software"</i> introduziu a ideia de que não devemos projetar por função, mas sim isolar as partes do sistema que têm maior probabilidade de mudar (volatilidade). | Representa o nível mais alto de abstração da reunião. Sai da "escovação de bits" e entra na estratégia de arquitetura, preparando os profissionais para projetarem sistemas resilientes a mudanças de mercado e de requisitos. |
| <b>A Inevitabilidade das Interfaces Agênticas</b> | A discussão sobre o fim das "telas ricas" e a transição para sistemas                                                                                                                             | É uma visão de futuro audaciosa. Remove o foco do desenvolvedor                                                                                                                                                                |

| <b>Título</b> | <b>O Que Foi Dito</b>                                                                    | <b>Justificativa</b>                                                                                                                                        |
|---------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               | baseados em chat, voz e agentes de IA, o que tornaria o CSS "menos relevante" no futuro. | da interface visual (front-end tradicional) e o coloca na inteligência do back-end, antecipando uma mudança na forma como usuários interagem com softwares. |

# PERGUNTAS E RESPOSTAS DO ENCONTRO



| Participante | Pergunta / Questionamento                                                                                                           | Resposta do Instrutor                                                                                                                                                                                                     | Análise da Resposta                                                                                                                            |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Igor         | Qual a maior dificuldade técnica ao adotar Minimal APIs, citando problemas com o Datadog?                                           | Gabriel explicou que, por serem baseadas em Lambdas e métodos de extensão, o rastreio e a criação de middlewares genéricos (como logs para Datadog) exigem uma implementação mais manual e menos convencional que no MVC. | <b>Satisfatória.</b> O instrutor validou o problema real de infraestrutura e apontou que a "mágica" do MVC às vezes faz falta em Minimal APIs. |
| Gustavo      | Ao colocar o Controller dentro do domínio para organizar o código, isso já seria o conceito de <i>Vertical Slice Architecture</i> ? | Gabriel esclareceu que colocar o controller no domínio é um passo, mas a <i>Vertical Slice</i> vai além: trata cada funcionalidade                                                                                        | <b>Satisfatória.</b> Corrigiu uma percepção comum, diferenciando "organização de pastas" de "estilo arquitetural".                             |



| Participante  | Pergunta / Questionamento                                                                                               | Resposta do Instrutor                                                                                                                                                                        | Análise da Resposta                                                                                                                        |
|---------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
|               |                                                                                                                         | como um recorte isolado de ponta a ponta.                                                                                                                                                    |                                                                                                                                            |
| <b>Rafael</b> | Faz mais sentido adotar Minimal API apenas quando o foco for microsserviços bem delimitados?                            | Gabriel alertou para o "medo" do termo microsserviço e enfatizou que o critério deve ser a maturidade do design: se o endpoint for enxuto e não vazar responsabilidade, Minimal API é ideal. | <b>Excelente.</b> Desmistificou a ideia de que tecnologia define arquitetura, focando na responsabilidade do código.                       |
| <b>Igor</b>   | Como lidar com a complexidade de arquiteturas como <i>Ports and Adapters</i> para membros do time que vêm do Front-end? | Gabriel admitiu que a carga cognitiva é alta e sugeriu que o design deve buscar simplificar a vida do desenvolvedor, prometendo uma aula específica para "derrubar                           | <b>Satisfatória (Promissória).</b> O instrutor acolheu a dor do líder de equipe, mas deixou o aprofundamento para o módulo de arquitetura. |

| Participante   | Pergunta / Questionamento                                                                                                    | Resposta do Instrutor                                                                                                                                                   | Análise da Resposta                                                                                                                                        |
|----------------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                |                                                                                                                              | verdades" sobre o tema.                                                                                                                                                 |                                                                                                                                                            |
| <b>Marcel</b>  | Estruturar com <i>Vertical Slice</i> seria basicamente criar uma classe para cada Action do Controller?                      | Gabriel confirmou que, na prática, cada Action vira um endpoint autônomo (classe específica), facilitando a manutenção isolada.                                         | <b>Satisfatória.</b><br>Resposta curta e direta que ajudou a materializar o conceito para o aluno.                                                         |
| <b>Eduardo</b> | É seguro usar um Singleton com <i>ConcurrentDictionary</i> para cache de dados que mudam pouco (2h de vida) em vez de Redis? | Gabriel sugeriu usar <i>IMemoryCache</i> (nativo), enquanto Thiago alertou que "mudar pouco" é um risco em produção e que o tempo de 2h é um valor arbitrário perigoso. | <b>Muito Satisfatória.</b> Os instrutores desencorajaram a "solução caseira" em favor de padrões nativos do .NET que já resolvem expiração e concorrência. |
| <b>Bruno</b>   | O .NET permite injetar um serviço Scoped em um Singleton?                                                                    | Houve um debate sobre as versões do .NET. Gabriel e Bruno concluíram que as versões                                                                                     | <b>Satisfatória.</b><br>Esclareceu a evolução do framework e a importância de                                                                              |

| Participante   | Pergunta / Questionamento                                                                                                           | Resposta do Instrutor                                                                                                                                              | Análise da Resposta                                                                                                                                |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
|                |                                                                                                                                     | novas (8+) já validam isso no build, mas que historicamente isso era um erro silencioso de produção.                                                               | ativar as flags de validação de escopo.                                                                                                            |
| <b>Eduardo</b> | O uso de <i>System.Threading.Channels</i> serviria para limitar a entrada de requisições (throttling) e evitar sobrecarga no banco? | Thiago Borba explicou que o ASP.NET já possui filas internas e que o problema de Eduardo parecia ser de bloqueio de I/O ou design, e não falta de uma fila manual. | <b>Profunda.</b> Evitou que o aluno implementasse uma complexidade desnecessária (Channels) para resolver um problema que o framework já gerencia. |
| <b>Rafael</b>  | O <i>Workflow Core</i> pode ser usado para gerenciar dependências de APIs externas que demoram a responder?                         | Gabriel confirmou que sim, pois o Workflow permite pausar o estado e aguardar eventos externos sem prender a thread da aplicação.                                  | <b>Satisfatória.</b> Apresentou uma ferramenta poderosa para resiliência que o aluno não conhecia.                                                 |
| <b>Gustavo</b> | Usar <i>Global Query Filters</i> para isolar                                                                                        | Gabriel concordou que é                                                                                                                                            | <b>Excelente.</b> Apontou o "ponto                                                                                                                 |

| Participante    | Pergunta / Questionamento                                                                                              | Resposta do Instrutor                                                                                                                                                  | Análise da Resposta                                                                                                                |
|-----------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                 | dados de Multi-tenancy é uma solução elegante?                                                                         | elegante para EF Core, mas alertou que, se o projeto usar Dapper ou SQL puro em algumas partes, a proteção do filtro é perdida e exige vigilância manual.              | cego" de uma solução que parece perfeita no papel, mas falha em arquiteturas híbridas.                                             |
| <b>Anderson</b> | Como o instrutor vê o uso do <i>LinqKit</i> para construir predicados dinâmicos e garantir o isolamento de inquilinos? | Gabriel validou o uso, mas mencionou o desafio de manter o time treinado para escrever LINQ complexo em vez de SQL puro, o que às vezes gera problemas de performance. | <b>Satisfatória.</b><br>Equilibrou a vantagem técnica da biblioteca com o desafio humano de manutenção do código.                  |
| <b>Bruno</b>    | Em uma aplicação Multi-tenant com uma única API, um cliente pesado pode prejudicar a                                   | Gabriel explicou que sim (problema do <i>Noisy Neighbor</i> ) e sugeriu usar Gateways para rotear clientes específicos para instâncias                                 | <b>Satisfatória.</b><br>Trouxe uma visão de infraestrutura e operações para resolver um problema que o código sozinho não resolve. |

| Participante  | Pergunta / Questionamento                                                      | Resposta do Instrutor                                                                                                                                                | Análise da Resposta                                                                                         |
|---------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
|               | performance dos outros?                                                        | exclusivas da mesma API.                                                                                                                                             |                                                                                                             |
| <b>Marcel</b> | Como seria o uso prático de um <i>Tenant Accessor</i> dentro de um Controller? | Gabriel mostrou no código que, em vez de acessar o Header manualmente, o desenvolvedor apenas injeta o Accessor e consome as propriedades do inquilino já validadas. | <b>Muito Satisfatória.</b> A demonstração de código sanou a dúvida sobre a "mágica" por trás do middleware. |

## AGORA É COM VOCÊ



Este resumo foi elaborado para apoiar a consolidação dos principais conceitos discutidos na sessão. No entanto, o verdadeiro valor deste workshop está em transformar conhecimento em prática.

- Aplique o que foi discutido
- Reflita sobre os insights e leve-os para seu contexto profissional
- Experimente, teste abordagens, aprenda com os resultados
- Adapte os aprendizados à realidade da sua equipe e dos seus projetos
- Evolua um pouco mais a cada sessão
- Quem pratica com intenção evolui com consistência.

## COMPARTILHE SUA JORNADA

Use o LinkedIn (ou outra rede profissional) para dividir reflexões, descobertas e experiências. Isso contribui para o seu posicionamento como alguém em constante evolução.

**MARQUE A @EXIMIA.CO**

*ele acompanha e interage com a comunidade de mentorandos.*



### **PARTICIPE DO GRUPO FECHADO NO WHATSAPP**

O grupo é seu canal direto com os instrutores e com outros participantes do Workshop em C# do Jeito Certo.

Utilize-o para esclarecer dúvidas, pedir opiniões, compartilhar resultados e fortalecer sua rede.

# CLUBE DE ESTUDOS



[elemarjr.com](http://elemarjr.com)